

两篇系统论文：缓存“怎么存”和“什么时候复用”

阅读起点：KV cache 是什么

自回归生成会反复用到前面 token 的 Key / Value。Prefill 先为输入建立 KV，Decode 再逐 token 读取旧 KV、追加新 KV。长输入和并发请求使这部分显存与重复计算都变得重要。以下两篇论文处理的是同一批 KV，但切入点不同。

SGLang / RadixAttention (NeurIPS 2024) [1]

问题：多轮对话、共享系统提示词、分叉生成会形成相同前缀。若每次都重新 Prefill，已算过的前缀 KV 被浪费。

做法：论文第 3 节把 token 序列与对应 KV 组织为 radix tree。新请求查找最长公共前缀，复用命中的 KV，再计算剩余部分；完成后插入新分支。缓存空间紧张时，从可淘汰的叶节点按 LRU 清理；正在被请求使用的节点由引用计数保护。调度器也会考虑匹配长度。论文实现的 KV 物理存储是非连续分页布局，每页对应一个 token。

我记住的区别：radix tree 管的是“哪个前缀与谁共享”；分页布局管的是“KV 在显存里怎么放”。图 3 的多轮聊天例子比单纯背定义更能看出树节点为何要分裂、保留和淘汰。

vLLM / PagedAttention (SOSP 2023) [2]

问题：输出长度预先不知道，连续预留一整段 KV 空间会留下未使用的槽位和内外碎片；显存装不下足够多的并发请求。

做法：把逻辑上连续的 token KV 映射到固定大小、物理上可不连续的块，用块表寻址，随生成过程按需分配。块还能在适当的请求 / 解码分支间共享。论文主要论证的是 KV 内存管理及其对 batch 容量和吞吐的影响。

对照来看，PagedAttention 偏重物理块分配与注意力读取，RadixAttention 偏重跨请求的前缀匹配和缓存生命周期。两者可配合使用。不能据 2023 年论文断言“vLLM 没有前缀缓存”：当前 vLLM 文档已描述基于哈希、只缓存完整块的 APC。[4]

Mooncake trace：数据字段、我的实验与结果边界

Mooncake FAST' 25 trace release [3]

发布页区分 FAST' 25 的 traces/ 与较早的 arxiv-trace/。我使用的是 traces/synthetic_trace.jsonl；发布页列出 3,993 条合成请求，并说明它来自公开数据集、采用 Poisson 到达。每行 JSONL 含 timestamp（相对到达毫秒）、input_length、output_length、hash_ids。hash_ids 表示重映射后的前缀块哈希，块大小为 512 token；相同 ID 可标示可复用前缀。文件不含原始文本，这是匿名化 trace，不是可直接发送给模型的 prompt。

与本地 12 条 workload 的对应

workload.py 读取文件前 12 行，用 input_length // 128 构造短提示词，限制目标输入为 32–320 个重复词，输出上限为 48 token；因此实际 input_tokens 会比目标值多出模板等 token。脚本没有照搬 trace 的 timestamp，而是用指数分布重新生成平均约 2 秒的请求间隔，并通过 Ray 发往 SGLang 的 HTTP 接口。CSV 记录了 trace 长度、目标长度、服务器返回的实际 token 数、状态码和 latency_s。现有 12 条结果均为 HTTP 200。

这组结果能说明什么，不能说明什么

它证明本地 SGLang 服务、请求生成和结果记录这一条链路跑通了。它不能证明 RadixAttention 带来了多少加速，也不是 Mooncake 原始负载的性能复现：输入 / 输出被缩放，原始 hash_ids 没有用于构造相同前缀，请求只有 12 条，也没有设置对照组或记录缓存命中率。若进一步研究，应固定模型和长度，分别发送共享前缀与无共享前缀的请求，再比较 TTFT、吞吐及缓存命中。

资料

- [1] Zheng et al., SGLang, NeurIPS 2024, Sec. 3.
- [2] Kwon et al., PagedAttention / vLLM, SOSP 2023, Sec. 3–4.
- [3] Mooncake FAST25-release README.
- [4] vLLM Docs, Automatic Prefix Caching.